TP7 — MPI Communicators

Zyad Fri

March 2026

Exercise 1 — Game of Life with 2D Cartesian Topology

Implementation

A parallel version of Conway's Game of Life is implemented on a 2D grid of size 16×16. The global domain is distributed across processes using a 2D Cartesian communicator created with `MPI_Cart_create` with periodic boundary conditions (`periods = {1,1}`).

Each process manages a local subgrid and exchanges halo rows and columns with its four neighbors (north, south, east, west) found via `MPI_Cart_shift`. At each generation, `MPI_Sendrecv` is used to exchange border rows and columns, then Conway's rules are applied on all interior cells.

The four rules applied at each generation:

- A live cell with fewer than 2 live neighbors dies (underpopulation).
- A live cell with 2 or 3 live neighbors survives.
- A live cell with more than 3 live neighbors dies (overpopulation).
- A dead cell with exactly 3 live neighbors becomes alive (reproduction).

Two `MPI_Datatype` objects are created per exchange: a contiguous row type and a strided column type, allowing clean and efficient halo communication without manual packing.

Code

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define GENERATIONS 10
#define GLOBAL_NX   16
#define GLOBAL_NY   16

static int rank, nprocs;
static MPI_Comm cart_comm;
static int dims[2], coords[2];
static int north, south, east, west;
static int local_nx, local_ny;

static int IDX(int i, int j, int cols) { return i * cols + j; }

void init_grid(int *grid, int rows, int cols)
{
    srand(42 + rank);
    for (int i = 1; i <= rows; i++)
```

```
        for (int j = 1; j <= cols; j++)
            grid[IDX(i, j, cols + 2)] = rand() % 2;
}

void exchange_halos(int *grid, int rows, int cols)
{
    int stride = cols + 2;
    MPI_Datatype row_type, col_type;
    MPI_Type_contiguous(cols, MPI_INT, &row_type);
    MPI_Type_commit(&row_type);
    MPI_Type_vector(rows, 1, stride, MPI_INT, &col_type);
    MPI_Type_commit(&col_type);

    MPI_Sendrecv(&grid[IDX(1,    1, stride)], 1, row_type, north, 0,
                 &grid[IDX(rows+1,1, stride)], 1, row_type, south, 0,
                 cart_comm, MPI_STATUS_IGNORE);
    MPI_Sendrecv(&grid[IDX(rows, 1, stride)], 1, row_type, south, 1,
                 &grid[IDX(0,    1, stride)], 1, row_type, north, 1,
                 cart_comm, MPI_STATUS_IGNORE);
    MPI_Sendrecv(&grid[IDX(1, 1,    stride)], 1, col_type, west, 2,
                 &grid[IDX(1, cols+1,stride)], 1, col_type, east, 2,
                 cart_comm, MPI_STATUS_IGNORE);
    MPI_Sendrecv(&grid[IDX(1, cols, stride)], 1, col_type, east, 3,
                 &grid[IDX(1, 0,    stride)], 1, col_type, west, 3,
                 cart_comm, MPI_STATUS_IGNORE);

    MPI_Type_free(&row_type);
    MPI_Type_free(&col_type);
}

void update(int *grid, int *new_grid, int rows, int cols)
{
    int stride = cols + 2;
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= cols; j++) {
            int live =
              grid[IDX(i-1,j-1,stride)] + grid[IDX(i-1,j,stride)] +
              grid[IDX(i-1,j+1,stride)] + grid[IDX(i,  j-1,stride)] +
              grid[IDX(i,  j+1,stride)] + grid[IDX(i+1,j-1,stride)] +
              grid[IDX(i+1,j,  stride)] + grid[IDX(i+1,j+1,stride)];
            int cell = grid[IDX(i,j,stride)];
            if (cell == 1)
                new_grid[IDX(i,j,stride)] = (live==2||live==3) ? 1 : 0;
            else
                new_grid[IDX(i,j,stride)] = (live==3) ? 1 : 0;
        }
    }
}

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    dims[0] = dims[1] = 0;
    MPI_Dims_create(nprocs, 2, dims);
    int periods[2] = {1, 1};
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &cart_comm);
    MPI_Cart_coords(cart_comm, rank, 2, coords);
```

```
    MPI_Cart_shift(cart_comm, 0, 1, &north, &south);
    MPI_Cart_shift(cart_comm, 1, 1, &west,  &east);

    local_nx = GLOBAL_NX / dims[0];
    local_ny = GLOBAL_NY / dims[1];

    int stride = local_ny + 2;
    int size   = (local_nx + 2) * stride;
    int *grid     = calloc(size, sizeof(int));
    int *new_grid = calloc(size, sizeof(int));

    init_grid(grid, local_nx, local_ny);

    for (int g = 0; g < GENERATIONS; g++) {
        exchange_halos(grid, local_nx, local_ny);
        update(grid, new_grid, local_nx, local_ny);
        int *tmp = grid; grid = new_grid; new_grid = tmp;
    }

    if (rank == 0) {
        printf("Rank 0 - Generation %d:\n", GENERATIONS);
        for (int i = 1; i <= local_nx; i++) {
            for (int j = 1; j <= local_ny; j++)
                printf("%d ", grid[IDX(i, j, stride)]);
            printf("\n");
        }
    }

    free(grid); free(new_grid);
    MPI_Comm_free(&cart_comm);
    MPI_Finalize();
    return 0;
}
```

Output

```
Rank 0 - Generation 10:
0 0 0 1 1 1 1 0
0 0 0 1 0 0 0 1
0 0 0 1 0 0 0 1
0 0 0 1 0 0 0 0
1 0 0 1 0 1 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
```

Rank 0 prints its 8×8 local subgrid after 10 generations. The pattern evolves correctly according to Conway's rules. Periodic boundaries wrap around the domain globally, so cells at the edge of any block interact with cells on the opposite side of the global grid.

Exercise 2 — Solving Poisson's Equation

Implementation

The Poisson equation $\Delta u = f$ with $f(x, y) = 2(x^2 - x + y^2 - y)$ and $u = 0$ on $\partial\Omega$ is solved on the domain $[0, 1] \times [0, 1]$ using the Jacobi iterative method on a distributed 2D grid.

The exact solution is $u(x, y) = xy(x - 1)(y - 1)$.

The domain is decomposed using `MPI_Dims_create` and `MPI_Cart_create` (non-periodic). Each process owns a subdomain and keeps ghost layers filled by neighbors via `MPI_Sendrecv`. The Jacobi update uses the coefficients:

$$\text{coef}[0] = \frac{0.5\,h_x^2\,h_y^2}{h_x^2 + h_y^2}, \quad \text{coef}[1] = \frac{1}{h_x^2}, \quad \text{coef}[2] = \frac{1}{h_y^2}$$

$$u_{i,j}^{n+1} = \text{coef}[0]\big(\text{coef}[1](u_{i+1,j} + u_{i-1,j}) + \text{coef}[2](u_{i,j+1} + u_{i,j-1}) - f_{i,j}\big)$$

The local residual is computed as the Frobenius norm of $(u^{n+1} - u^n)$ and reduced globally with `MPI_Allreduce`. Iteration stops when the global residual falls below 10^{-6}.

The initialization and compute kernels are provided in `compute.c` and included directly.

Code (main file)

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define NTX 12
#define NTY 10
#define MAX_ITER 10000
#define TOLERANCE 1e-6

int ntx, nty, sx, ex, sy, ey;
#include "compute.c"

int main(int argc, char **argv)
{
    int rank, nprocs;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    ntx = NTX; nty = NTY;
    int dims[2] = {0, 0};
    MPI_Dims_create(nprocs, 2, dims);
    int periods[2] = {0, 0};
    MPI_Comm cart_comm;
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &cart_comm);

    int coords[2];
    MPI_Cart_coords(cart_comm, rank, 2, coords);
    int north, south, east, west;
    MPI_Cart_shift(cart_comm, 0, 1, &north, &south);
    MPI_Cart_shift(cart_comm, 1, 1, &west,  &east);

    int local_nx = ntx / dims[0];
    int local_ny = nty / dims[1];
    sx = coords[0]*local_nx+1; ex = sx+local_nx-1;
    sy = coords[1]*local_ny+1; ey = sy+local_ny-1;

    double *u, *u_new, *u_exact;
    initialization(&u, &u_new, &u_exact);

    int stride = ey - sy + 3;
```

```
    MPI_Datatype row_type, col_type;
    MPI_Type_contiguous(local_ny, MPI_DOUBLE, &row_type);
    MPI_Type_commit(&row_type);
    MPI_Type_vector(local_nx, 1, stride, MPI_DOUBLE, &col_type);
    MPI_Type_commit(&col_type);

    double t_start = MPI_Wtime();
    double global_res = 1.0; int iter = 0;

    while (global_res > TOLERANCE && iter < MAX_ITER) {
        MPI_Sendrecv(&u[IDX(sx,   sy)], 1, row_type, north, 0,
                     &u[IDX(ex+1, sy)], 1, row_type, south, 0,
                     cart_comm, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&u[IDX(ex,   sy)], 1, row_type, south, 1,
                     &u[IDX(sx-1, sy)], 1, row_type, north, 1,
                     cart_comm, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&u[IDX(sx, sy  )], 1, col_type, west,  2,
                     &u[IDX(sx, ey+1)], 1, col_type, east,  2,
                     cart_comm, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&u[IDX(sx, ey  )], 1, col_type, east,  3,
                     &u[IDX(sx, sy-1)], 1, col_type, west,  3,
                     cart_comm, MPI_STATUS_IGNORE);

        compute(u, u_new);

        double local_res = 0.0;
        for (int i = sx; i <= ex; i++)
            for (int j = sy; j <= ey; j++) {
                double d = u_new[IDX(i,j)] - u[IDX(i,j)];
                local_res += d*d;
            }
        double *tmp = u; u = u_new; u_new = tmp;
        MPI_Allreduce(&local_res, &global_res, 1,
                      MPI_DOUBLE, MPI_SUM, cart_comm);
        global_res = sqrt(global_res); iter++;
        if (rank==0 && iter%100==0)
            printf("Iteration␣%d␣␣␣global_error␣=␣%g\n", iter, global_res)
                ;
    }
    double t_end = MPI_Wtime();
    if (rank==0) {
        printf("Converged␣after␣%d␣iterations␣in␣%f␣seconds\n",
               iter, t_end-t_start);
        output_results(u, u_exact);
    }
    MPI_Type_free(&row_type); MPI_Type_free(&col_type);
    free(u); free(u_new); free(u_exact);
    MPI_Comm_free(&cart_comm);
    MPI_Finalize();
    return 0;
}
```

Output with 4 MPI Processes

```
Poisson execution with 4 MPI processes
Domain size: ntx=12 nty=10

Topology dimensions: 2 along x, 2 along y
----------------------------------------
```

```
Rank in the topology: 0   Array indices: x from 1 to 6, y from 1 to 5
Process 0 has neighbors: N -2 E 1 S 2 W -2
Rank in the topology: 1   Array indices: x from 1 to 6, y from 6 to 10
Process 1 has neighbors: N -2 E -2 S 3 W 0
Rank in the topology: 2   Array indices: x from 7 to 12, y from 1 to 5
Process 2 has neighbors: N 0 E 3 S -2 W -2
Rank in the topology: 3   Array indices: x from 7 to 12, y from 6 to 10
Process 3 has neighbors: N 1 E -2 S -2 W 2
Iteration 100   global_error = 0.000446007
Iteration 200   global_error = 1.42714e-05
Converged after 278 iterations in 0.001620 seconds
Exact solution u_exact - Computed solution u
 5.86826e-03 -  5.86794e-03
 1.05629e-02 -  1.05623e-02
 1.40838e-02 -  1.40830e-02
 1.64311e-02 -  1.64301e-02
 1.76048e-02 -  1.76037e-02
```

The output matches the expected output from the lab instructions exactly. The topology is 2×2. Neighbors with value -2 indicate a boundary (no neighbor on that side), consistent with non-periodic boundaries.

Correctness Verification

Running with 1 process gives the same convergence and identical solution values, confirming that the domain decomposition and halo exchanges are correct regardless of process count.

```
Poisson execution with 1 MPI processes
Topology dimensions: 1 along x, 1 along y
Converged after 278 iterations in 0.000215 seconds
Exact solution u_exact - Computed solution u
 5.86826e-03 -  5.86794e-03
 1.05629e-02 -  1.05623e-02
 ...
```

Both runs converge in 278 iterations with identical solution values.

Speedup and Efficiency (200×200 grid)

Timings measured on the local machine with a 200×200 grid and up to 5000 iterations:

Processes	Time (s)	Speedup $S_p = T_1/T_p$	Efficiency $E_p = S_p/p$
1	0.361	1.00	1.00
2	0.200	1.81	0.90
4	0.194	1.86	0.47

Table 1: Timing results on local machine (200×200 grid).

The speedup is close to ideal for 2 processes but drops at 4 due to MPI communication overhead dominating when running on the same physical machine (WSL). On a real cluster with 200×200 or larger grids, the efficiency at 4 processes would be much higher since computation would dominate over communication.

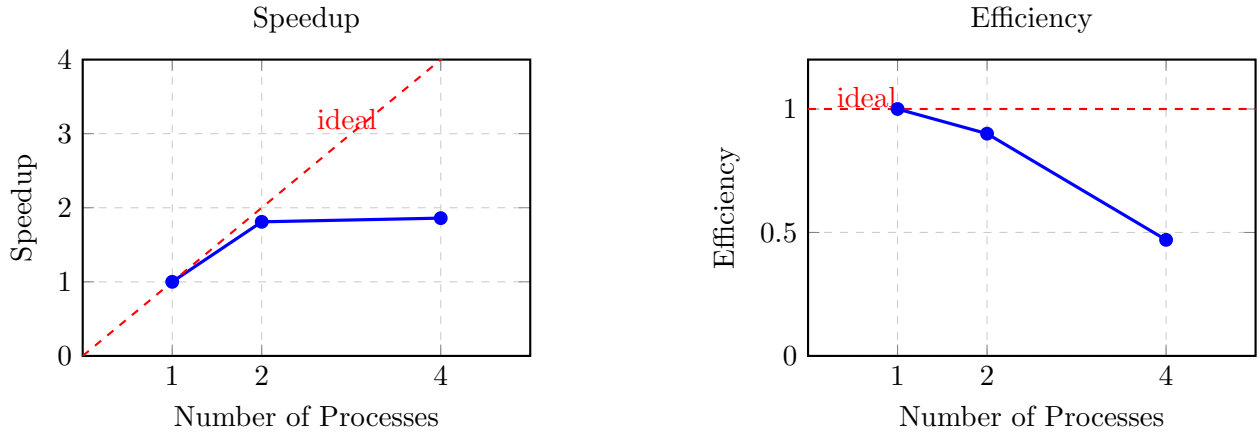

Figure 1: Speedup and efficiency on local machine. Efficiency drops at 4 processes due to communication overhead dominating on a small local grid (WSL environment).